The Journal of Open Source Software

Multi-Device UART Communication with the dwiBus Library

Ali Çimen [1¶]

1 Yönetim Bilişim Sistemleri Bölümü, İstanbul Gelişim Üniversitesi, İstanbul, Türkiye ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 10 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The **dwiBus** library provides a modern, reliable communication protocol for multi-device UART communication on shared buses, catering specifically to Arduino and ESP32 platforms. Designed to overcome the limitations of traditional protocols like Modbus RTU, it integrates CRC-based error checking, scalability for up to 255 devices, and a flexible API for seamless integration in IoT and embedded systems.

Statement of Need

As industrial automation systems become more IoT-enabled, effective low-level communication protocols are essential. Classical systems like Modbus RTU are widely adopted but exhibit constraints that hinder their suitability for modern IoT applications. The **dwiBus** library addresses these challenges, offering enhanced reliability, scalability, and flexibility.

Challenges with Traditional Protocols

1. **Incompatibility with Modern Wireless Systems**
   Modbus RTU cannot directly integrate with technologies like Zigbee or Wi-Fi, limiting its use in distributed IoT systems (Gandhi & Adhyaru, 2015).
2. **Susceptibility to Electromagnetic Interference (EMI)**
   Modbus RTU lacks robust error detection and correction, making it unreliable in noisy environments (Urrea et al., 2016).
3. **Scalability Issues**
   Modbus RTU supports up to 247 devices in theory, but performance degrades significantly beyond 5-10 devices per segment (Guarese et al., 2012).

The Role of dwiBus

The **dwiBus** library is a modern alternative designed to mitigate these issues, with features tailored for IoT and industrial automation.

Key Features

- **Enhanced Data Integrity:** CRC-8 error checking protects against EMI and ensures reliable communication (Urrea & Morales, 2019).
- **Scalability:** Supports up to 255 devices with unique addressing.

- **Broadcast Messaging:** Enables efficient communication across distributed networks.

- **Flexibility:** Easily integrates with Arduino and ESP32 platforms, supporting both HardwareSerial and SoftwareSerial interfaces.

Usage

The **dwiBus** library provides a structured and reliable UART communication protocol for multi-device networks. Designed for Arduino and ESP32 platforms, it simplifies data transmission between up to 255 devices with CRC error checking, packet management, and broadcast messaging capabilities.

Setup and Initialization

To use the dwiBus library, the user must include the library in their project and initialize it with the appropriate communication interface. Each device in the network is assigned a unique address, which serves as its identifier.

```
#include <dwiBus.h>

// Create a dwiBus instance
dwiBus bus;

// Define a unique address for this device
const uint8_t DEVICE_ADDRESS = 1;

void setup() {
    Serial.begin(9600);              // Start serial communication
    bus.begin(&Serial, DEVICE_ADDRESS); // Initialize dwiBus with HardwareSerial and add
    Serial.println("Device initialized!");
}
```

Sending and Receiving Messages

The dwiBus library provides straightforward functions for sending and receiving messages between devices. A message can be sent to a specific device using its unique address or broadcasted to all devices in the network.

```
bus.write(receiverAddress, "Hello Device");
```

Broadcasting Messages

```
bus.broadcast("System Check");
```

Receiving Messages

```
if (bus.available()) {
    uint8_t sender;
    String message = bus.read(&sender);
    Serial.print("Message from Device ");
    Serial.print(sender);
    Serial.print(": ");
    Serial.println(message);
}
```

DRAFT

Integration with HardwareSerial and SoftwareSerial

The library is compatible with both HardwareSerial and SoftwareSerial, allowing developers to select the interface that best suits their application. This flexibility ensures compatibility with a wide range of hardware platforms, including Arduino and ESP32.

```
#include <SoftwareSerial.h>
#include <dwiBus.h>

// Initialize SoftwareSerial
SoftwareSerial softSerial(10, 11); // RX, TX
dwiBus bus;

void setup() {
    softSerial.begin(9600);
    bus.begin(&softSerial, 2); // Initialize dwiBus with SoftwareSerial and address
}
```

Advanced Usage: Modular Integration

The library's modular API allows developers to incorporate it into complex systems with minimal changes to their existing codebase. The clear separation of initialization, messaging, and error-checking logic makes the library easy to integrate into new and legacy systems alike.

References

Gandhi, P., & Adhyaru, D. M. (2015). Critical data reliability issues with modbus protocol in smart grid application. *Proceedings of the 2015 International Conference on Energy Systems and Applications*, 1–5. https://doi.org/10.1109/EESCO.2015.7253788

Guarese, G. B. M., Sieben, F. G., Webber, T., Dillenburg, M. R., & Marcon, C. (2012). Exploiting modbus protocol in wired and wireless multilevel communication architecture. *Proceedings of the 2012 Brazilian Symposium on Computing System Engineering*, 13–18. https://doi.org/10.1109/SBESC.2012.12

Urrea, C., & Morales, C. (2019). Enhancing modbus RTU communications for smart metering applications. *International Journal of Smart Grid and Clean Energy*, *8*(2), 123–130. https://doi.org/10.1155/2019/7010717

Urrea, C., Morales, C., & Kern, J. (2016). Implementation of error detection and correction in modbus RTU for industrial applications. *International Journal of Critical Infrastructure Protection*, *15*, 27–37. https://doi.org/10.1016/j.ijcip.2016.07.001

DRAFT